Diagrama de classes do Prestador de Serviço

O que o diagrama mostra, como ler cada símbolo e o que ele revela sobre o projeto. Versão da main de 06/11/2026

1. O que é um diagrama de classes

O diagrama de classes é o diagrama estrutural mais usado da UML (Unified Modeling Language). Ele mostra a estrutura estática do sistema: quais classes existem, o que cada uma guarda (atributos), o que cada uma faz (métodos) e como elas se relacionam (herança, associação, dependência).

Ele é "estático" porque não mostra tempo nem ordem de execução. O diagrama não diz que o contratante primeiro cria a avaliação e depois o prestador recebe. Ele só diz que essas classes existem e estão ligadas. Ordem de chamadas é papel do diagrama de sequência, e mudança de estado de um objeto é papel do diagrama de estados.

Para que serve no desenvolvimento

- Comunicar a arquitetura: alguém novo no time entende como o sistema está dividido sem abrir 13 arquivos
- Projetar antes de codar: discutir onde fica cada responsabilidade antes de escrever código
- Documentar depois de codar: é o caso deste diagrama, que foi gerado a partir do código que já existe
- Enxergar problemas de projeto: classe que faz coisa demais, dependência que não deveria existir, ligação que falta

Um diagrama que mostra testes

A UML não tem um diagrama próprio para testes. Existe uma extensão oficial da OMG, o UML Testing Profile (UTP), mas ela é pouco usada no ensino. Na prática, há três formas comuns de representar testes:

Forma	O que mostra	Quando usar
Diagrama de classes com as classes de teste	Qual classe de teste verifica qual classe de produção (rastreabilidade)	Ver o que está coberto e o que ficou sem teste. É o deste documento
Pirâmide de testes	Quantidade de testes de unidade, integração e sistema	Avaliar se a proporção está saudável (aula 4)
Diagrama de sequência de um teste	A ordem das chamadas dentro de um cenário, incluindo o mock	Explicar um teste com `verify` ou um fluxo entre várias classes

Neste diagrama, as classes de teste foram desenhadas como classes normais, marcadas com o estereótipo `<<test>>`. Esse estereótipo não é padrão da UML: ele foi criado para este diagrama. A UML permite isso, porque estereótipos servem para estender a linguagem.

2. O diagrama

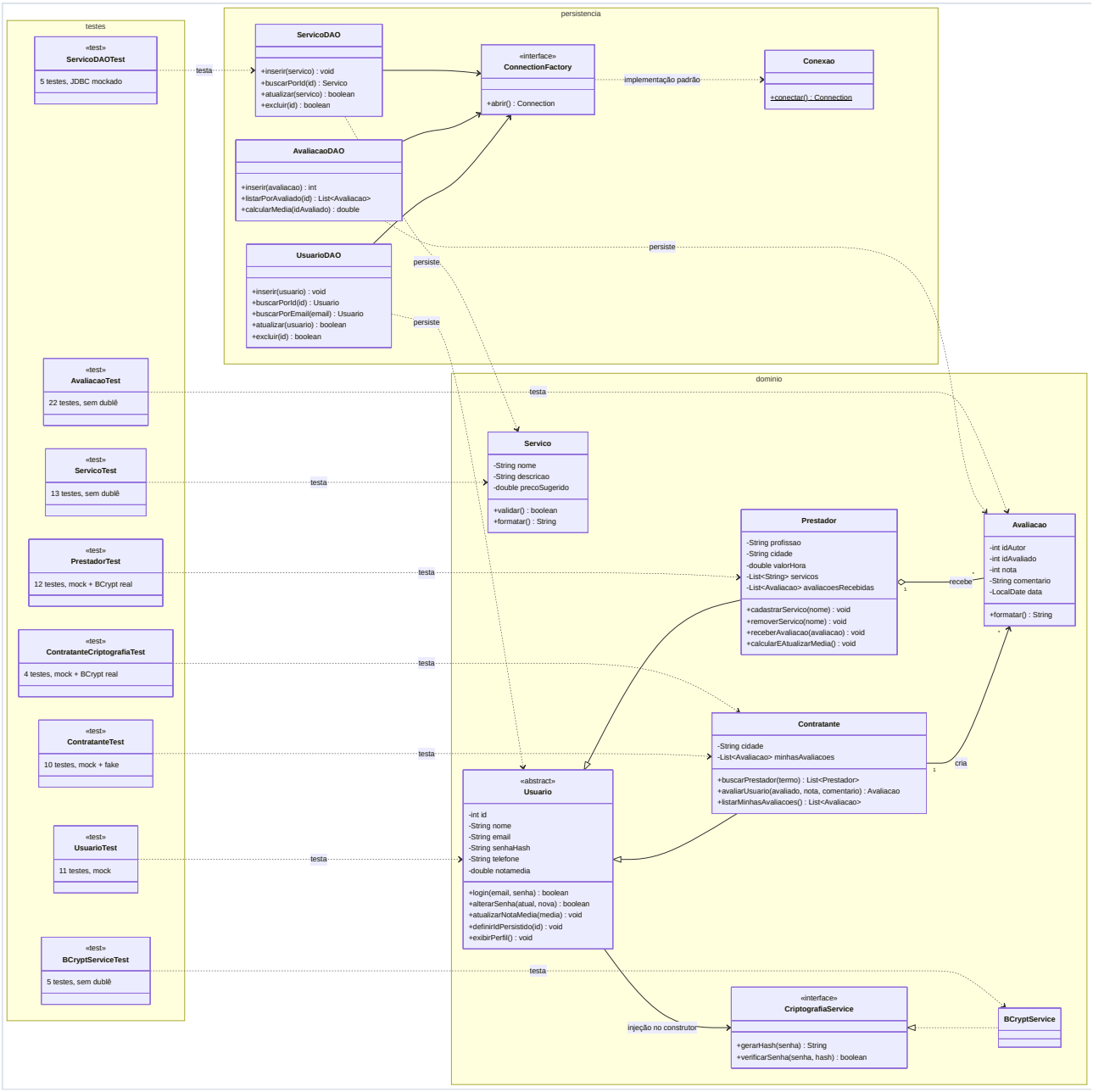


Diagrama de classes do Prestador de Serviço, gerado em Mermaid a partir da main de 06/11. O código está no apêndice.

Ele tem três blocos, chamados pacotes (no Mermaid, `namespace`): dominio, persistencia e testes.

3. Como ler uma classe

Cada caixa é uma classe, dividida em três compartimentos:

1. Nome, com o estereótipo em cima quando houver
2. Atributos: os dados que a classe guarda
3. Métodos: o que a classe faz, com os parâmetros entre parênteses e o tipo de retorno depois

Símbolos dentro da caixa

Símbolo	Significado	Exemplo no projeto
`-`	private	`-String senhaHash` no `Usuario`: só a própria classe acessa. É o encapsulamento que impede alguém de trocar o hash por fora
`+`	public	`+login(email, senha) boolean`: qualquer classe pode chamar
`#`	protected	Não aparece no diagrama, mas o construtor novo do `Usuario` (com `senhaJaCriptografada`) é protected: só as subclasses usam
`$`	static (sublinhado na imagem)	`+conectar()$ Connection`: chama-se `Conexao.conectar()`, sem criar objeto
`~T~`	tipo genérico	`List~Avaliacao~` é o Mermaid escrevendo `List<Avaliacao>`
`<<abstract>>`	classe abstrata	`Usuario`: não pode ser instanciada. Ninguém cria um "usuário", cria um `Contratante` ou um `Prestador`
`<<interface>>`	interface	`CriptografiaService` e `ConnectionFactory`: só dizem quais métodos existem, sem implementação

4. Os relacionamentos (as setas)

Relacionamento	Desenho	No Mermaid	No projeto
Herança (generalização)	Linha cheia, triângulo vazado na classe-mãe	`<|--`	`Contratante` e `Prestador` são um `Usuario` e herdam `login`, `alterarSenha` e `atualizarNotaMedia`
Realização	Linha tracejada, triângulo vazado na interface	`<|..`	`BCryptService` implementa `CriptografiaService`
Associação	Linha cheia com seta	`-->`	`Usuario` guarda um `CriptografiaService` como atributo. Os DAOs guardam uma `ConnectionFactory`

Agregação	Linha cheia, losango vazado no "todo"	`o--`	`Prestador` tem uma lista de `Avaliacao`, mas a avaliação existe por conta própria
Composição	Linha cheia, losango cheio no "todo"	`*--`	Não usada. Seria para uma parte que morre junto com o todo
Dependência	Linha tracejada com seta	`..>`	Uma classe usa a outra sem guardá-la como atributo. `UsuarioDAO ..> Usuario` (persiste) e todas as setas "testa"

Multiplicidade

Os números nas pontas das setas dizem quantos objetos de cada lado participam da relação:

- `1`: exatamente um. `*`: zero ou mais. `0..1`: nenhum ou um. `1..*`: pelo menos um
- `Contratante "1" --> "*" Avaliacao`: um contratante cria várias avaliações, e cada avaliação tem um autor
- `Prestador "1" o-- "*" Avaliacao`: um prestador recebe várias avaliações

Rótulo da seta

O texto no meio da seta ("cria", "recebe", "persiste", "testa", "injeção no construtor") nomeia o papel da relação. Ele não muda o tipo da seta, só deixa a leitura mais rápida.

> Confusões comuns em prova
>
> - Agregação × composição: na composição, a parte não existe sem o todo (excluir uma casa exclui os cômodos). Na agregação, a parte sobrevive (excluir um time não exclui os jogadores). Avaliação de prestador é agregação porque, no banco, a avaliação continua existindo e tem autor próprio
> - Associação × dependência: associação é quando a classe guarda a outra num atributo. Dependência é quando só usa num método (parâmetro, variável local, retorno)
> - Herança × realização: herança é entre classes (`extends`), realização é de classe para interface (`implements`)
> - A seta da herança aponta para a classe-mãe, e não para a filha

5. Pacote dominio

É onde ficam as regras de negócio, sem nada de banco de dados.

Classe	Responsabilidade
`Usuario`	Classe abstrata com o que todo usuário tem: dados de contato, senha em hash, nota média, login e troca de senha. Recebe um `CriptografiaService` no construtor

`Contratante`	Usuário que contrata. Tem cidade, cria avaliações (`avaliarUsuario`) e guarda as que fez. `buscarPrestador` e `listarMinhasAvaliacoes` ainda devolvem lista vazia
`Prestador`	Usuário que oferece serviço. Tem profissão, cidade, valor por hora e lista de serviços (por nome). Recebe avaliações e recalcula a média
`Avaliacao`	Nota de 1 a 5, comentário opcional de até 500 caracteres e data que não pode ser futura. Valida tudo no construtor e lança `IllegalArgumentException` com todos os erros juntos
`Servico`	Serviço com nome, descrição e preço sugerido não negativo. Ainda não está ligado ao `Prestador`, que guarda os serviços como texto
`CriptografiaService`	Interface com `gerarHash` e `verificarSenha`
`BCryptService`	Implementação real da interface, com a biblioteca jBCrypt

Por que a interface CriptografiaService importa

O `Usuario` depende da interface, e não do `BCryptService` direto. É o princípio da inversão de dependência (o "D" do SOLID): depender da abstração, não da implementação. Na prática, isso permite que o teste injete um mock ou um fake no lugar do BCrypt. O `ContratanteTest` usa um `CriptografiaFake` que devolve "hash:" + senha, e o `UsuarioTest` usa um mock do Mockito. Sem essa interface, todo teste de usuário precisaria do BCrypt de verdade.

6. Pacote persistencia

Leva os objetos do domínio para o banco MySQL e traz de volta. Segue o padrão DAO (Data Access Object): uma classe por entidade, com as operações de CRUD (criar, ler, atualizar, excluir), para que o domínio não tenha SQL misturado.

Classe	Responsabilidade
`UsuarioDAO`	CRUD de usuários. Na leitura, decide se devolve um `Contratante` ou um `Prestador` e usa o construtor com `senhaJaCriptografada`
`AvaliacaoDAO`	Grava e lista avaliações por autor ou por avaliado, e calcula a média no banco
`ServicoDAO`	CRUD de serviços
`ConnectionFactory`	Interface com um método só, `abrir()`, que devolve uma conexão
`Conexao`	Abre a conexão real com o MySQL, lendo os dados de acesso das variáveis de ambiente. É a implementação padrão: o construtor vazio dos DAOs usa `Conexao::conectar`

É a mesma ideia da criptografia: os DAOs dependem da interface `ConnectionFactory`, e não da `Conexao`. Por isso o `ServicoDAOTest` consegue testar o DAO sem banco nenhum, passando uma fábrica que devolve uma `Connection` mockada.

7. Pacote testes

Cada classe de teste aponta, com a seta "testa", para a classe de produção que ela verifica. Dentro da caixa está a quantidade de testes e o tipo de dublê usado.

Classe de teste	Testes	Dublês	Classificação
`AvaliacaoTest`	22	Nenhum	Unidade, sem dublê (a classe não tem dependências)
`ServicoTest`	13	Nenhum	Unidade, sem dublê
`BCryptServiceTest`	5	Nenhum	Unidade, sem dublê
`PrestadorTest`	12	Mock + BCrypt real	Unidade, com mock
`UsuarioTest`	11	Mock	Unidade, com mock
`ContratanteTest`	10	Mock + fake	Unidade, com mock
`ContratanteCriptografiaTest`	4	Mock + BCrypt real	Unidade, com mock
`ServicoDAOTest`	5	JDBC mockado	Unidade, com mock (não toca no banco)
Total	**82**		

Tipos de dublê de teste

- Mock: objeto falso criado pelo Mockito, programado com `when(...).thenReturn(...)`, e que permite conferir chamadas com `verify`
- Fake: implementação simples e funcional, escrita à mão, como o `CriptografiaFake`
- Sem dublê: a classe testada não depende de nada, então o teste usa só objetos reais e continua sendo unitário

8. O que o diagrama revela sobre o projeto

Classes sem teste

Uma classe de produção que não recebe nenhuma seta "testa" está sem teste: `UsuarioDAO`, `AvaliacaoDAO`, `Conexao` e `Main`. Os dois DAOs entraram em 05/11 e são as maiores classes do projeto (o `UsuarioDAO` tem mais de 500 linhas). Com o stage de Cobertura exigindo 70% de linhas, eles podem derrubar a cobertura total e reprovar o build.

A pirâmide só tem a base

Os 82 testes são de unidade. Não há teste de integração (várias classes reais juntas, ou o DAO falando com um banco de verdade) nem teste de sistema. Pela recomendação do livro do Valente, o ideal fica perto de 70% unidade, 20% integração e 10% sistema. Ter só testes de unidade significa que cada peça funciona sozinha, mas ninguém verificou se elas funcionam juntas.

O bug do avaliarUsuario aparece no desenho

> Repare nas setas que chegam na `Avaliacao`: "cria" vem do `Contratante` e "recebe" vem do `Prestador`, mas não existe nenhuma ligação entre `Contratante` e `Prestador`. No código acontece o mesmo: o `avaliarUsuario` cria a avaliação e guarda na lista do contratante, mas nunca chama `receberAvaliacao` do prestador. A nota não chega ao prestador sozinha. Se o fluxo estivesse completo, haveria uma dependência `Contratante ..> Prestador` (o contratante chamaria um método do prestador).

Inconsistências com os requisitos

- `Prestador` guarda `List~String~ servicos` e um único `valorHora`, sem ligação com a classe `Servico`. O RF05 pede "os respectivos valores cobrados" de cada serviço
- Não há nenhuma classe "Contratação" ou "Pedido". O RF08 pede avaliação "após a realização de um serviço", e o diagrama mostra que o sistema não tem como saber se um serviço aconteceu

9. O que o diagrama não mostra

Todo diagrama é uma simplificação, escolhida para o que se quer comunicar. Neste:

- Getters e setters foram omitidos, e alguns atributos também (`id` da `Avaliacao` e do `Servico`, `descricao` do `Prestador`)
- Os construtores não aparecem, inclusive o construtor com `senhaJaCriptografada`
- Nos DAOs aparecem só os métodos principais (faltam `listarTodos`, `buscarPorNome`, `listarPorAvaliador` etc.)
- Os parâmetros dos métodos aparecem só pelo nome, sem tipo
- A classe `Main` ficou de fora, porque só monta um exemplo com o `ServicoDAO`
- Os testes estão agrupados por classe, sem listar cada método de teste

10. Como gerar e editar

O diagrama foi escrito em Mermaid, uma linguagem de texto que vira desenho. A vantagem sobre desenhar à mão é que o diagrama fica versionado no Git junto com o código, e qualquer mudança aparece no diff.

- Ver na hora: colar o código em mermaid.live
- No GitHub: colocar num arquivo `.md` dentro de um bloco que começa com ```` ```mermaid ```` e termina com ```` ``` ````. O GitHub desenha sozinho, então daria para colocar na pasta `documentacao`
- Exportar imagem: em mermaid.live, ou pelo terminal com o Mermaid CLI (`mmdc -i diagrama.mmd -o diagrama.png`)

Sintaxe usada

Trecho	O que faz

`classDiagram`	Declara que é um diagrama de classes
`direction LR`	Organiza da esquerda para a direita
`namespace dominio { ... }`	Agrupa classes num pacote
`class Usuario { ... }`	Declara a classe, com atributos e métodos dentro
`<<abstract>>` na primeira linha	Estereótipo
`+metodo(param) Retorno`	Método: visibilidade, nome, parâmetros e tipo de retorno
`A "1" --> "*" B : rótulo`	Relação com multiplicidade e rótulo

> Para revisar
>
> - Diagrama de classes = estrutura estática. Diagrama de sequência = ordem das chamadas no tempo
> - Os três compartimentos: nome, atributos, métodos. Visibilidade: + public, - private, # protected
> - Seis relações: herança, realização, associação, agregação, composição, dependência
> - Interface no meio da dependência (CriptografiaService, ConnectionFactory) = código testável com mock
> - Classe sem seta "testa" = classe sem teste

Apêndice: código Mermaid

```
classDiagram
    direction LR

    namespace dominio {
        class Usuario {
            <<abstract>>
            -int id
            -String nome
            -String email
            -String senhaHash
            -String telefone
            -double notamedia
            +login(email, senha) boolean
            +alterarSenha(atual, nova) boolean
            +atualizarNotaMedia(media) void
            +definirIdPersistido(id) void
            +exibirPerfil() void
        }
        class Contratante {
            -String cidade
            -List~Avaliacao~ minhasAvaliacoes
            +buscarPrestador(termo) List~Prestador~
            +avaliarUsuario(avaliado, nota, comentario) Avaliacao
            +listarMinhasAvaliacoes() List~Avaliacao~
        }
        class Prestador {
            -String profissao
            -String cidade
            -double valorHora
            -List~String~ servicos
            -List~Avaliacao~ avaliacoesRecebidas
            +cadastrarServico(nome) void
            +removerServico(nome) void
            +receberAvaliacao(avaliacao) void
            +calcularEAtualizarMedia() void
        }
        class Avaliacao {
            -int idAutor
            -int idAvaliado
            -int nota
            -String comentario
            -LocalDate data
            +formatar() String
        }
        class Servico {
            -String nome
            -String descricao
            -double precoSugerido
            +validar() boolean
            +formatar() String
        }
        class CriptografiaService {
            <<interface>>
            +gerarHash(senha) String
            +verificarSenha(senha, hash) boolean
        }
        class BCryptService
    }

    namespace persistencia {
        class UsuarioDAO {
            +inserir(usuario) void
            +buscarPorId(id) Usuario
            +buscarPorEmail(email) Usuario
            +atualizar(usuario) boolean
            +excluir(id) boolean
        }
```

```
        class AvaliacaoDAO {
            +inserir(avaliacao) int
            +listarPorAvaliado(id) List~Avaliacao~
            +calcularMedia(idAvaliado) double
        }
        class ServicoDAO {
            +inserir(servico) void
            +buscarPorId(id) Servico
            +atualizar(servico) boolean
            +excluir(id) boolean
        }
        class ConnectionFactory {
            <<interface>>
            +abrir() Connection
        }
        class Conexao {
            +conectar()$ Connection
        }
    }

    namespace testes {
        class UsuarioTest {
            <<test>>
            11 testes, mock
        }
        class ContratanteTest {
            <<test>>
            10 testes, mock + fake
        }
        class ContratanteCriptografiaTest {
            <<test>>
            4 testes, mock + BCrypt real
        }
        class PrestadorTest {
            <<test>>
            12 testes, mock + BCrypt real
        }
        class AvaliacaoTest {
            <<test>>
            22 testes, sem dublê
        }
        class ServicoTest {
            <<test>>
            13 testes, sem dublê
        }
        class BCryptServiceTest {
            <<test>>
            5 testes, sem dublê
        }
        class ServicoDAOTest {
            <<test>>
            5 testes, JDBC mockado
        }
    }

    Usuario <|-- Contratante
    Usuario <|-- Prestador
    CriptografiaService <|.. BCryptService
    Usuario --> CriptografiaService : injeção no construtor
    Contratante "1" --> "*" Avaliacao : cria
    Prestador "1" o-- "*" Avaliacao : recebe
    UsuarioDAO ..> Usuario : persiste
    AvaliacaoDAO ..> Avaliacao : persiste
    ServicoDAO ..> Servico : persiste
    UsuarioDAO --> ConnectionFactory
    AvaliacaoDAO --> ConnectionFactory
    ServicoDAO --> ConnectionFactory
    ConnectionFactory ..> Conexao : implementação padrão

    UsuarioTest ..> Usuario : testa
    ContratanteTest ..> Contratante : testa
    ContratanteCriptografiaTest ..> Contratante : testa
    PrestadorTest ..> Prestador : testa
```

```
    AvaliacaoTest ..> Avaliacao : testa
    ServicoTest ..> Servico : testa
    BCryptServiceTest ..> BCryptService : testa
    ServicoDAOTest ..> ServicoDAO : testa
```